Base64 uses the upper-case and lower-case letters (A-Z and a-z), decimal digits (0-9), and two extra characters to encode 64 different values. In the most common variant, these two extra characters are + and / for the 63rd and 64th values respectively:

0 = A, 1 = B, 2 = C, …, 25 = Z, 26 = a, …, 51 = z, 52 = 0, …, 61 = 9, 62 = +, 63 = /

For example, the ASCII string “Hello” is the 5-byte sequence 72, 101, 108, 108, 111 in decimal form, or in binary: 01001000 (H), 01100101 (e), 01101100 (l), 01101100 (l), 01101111 (o). When Base64-encoded this results in the string “SGVsbG8=”, where S is the encoding of the first 6 bits (010010), G is the encoding of the next 6 bits (000110), and so on. The equals sign at the end indicates that the input wasn’t an exact multiple of 6 bits and so has been padded with two zero bits, resulting in the final value 111100, which encodes to the character “8”. There can be either zero, one, or two equals signs at the end depending on the amount of padding that was required to make the input a multiple of 6 bits.

As well as the normal Base64-encoding scheme we have just described, there is also an URL-safe scheme in which the + character is replaced with -, and the / character with _, as these characters can be used in a URL without needing extra encoding. It is common in this scheme to also omit the padding characters, as the padding can be always calculated by looking at the length of the encoded string.

While Base64-encoded data looks like gibberish, it is easily reversible using widely available tools such as https://www.base64decode.org or the UNIX `base64 -d` command.

## 3.3.2 Secure password storage with Scrypt

Web browsers have built-in support for HTTP Basic authentication (albeit with some quirks that you’ll see later), as does curl and many other command-line tools. This allows us to easily send a username and password to the API, but you need to securely store and validate that password. This is a more complex topic than it first appears, which you’ll cover in depth in later chapters. For the purposes of this example you’re going to use a library that takes care of most of the details for us using the *Scrypt* secure password storage algorithm. Other secure password